Vulnerability Report For Web Application

Issue 1 >> Frontend and Backend API Connection

When we started this project, the frontend and backend were not properly connected. Because of this, the application was not able to correctly exchange data between the client side and server side.

This was fixed by properly setting up and testing the API integration between both sides. After the changes, the frontend is now successfully communicating with the backend and data flow works as expected.

Issue 2 >> Inventory Feature and CRUD Operations

There was a problem with the product functionality in the system. The product features were incomplete and some operations were not working properly.

To fix this, full CRUD operations (Create, Read, Update, Delete) were implemented for products and inventory. And now the inventory system is fully functional and allowed proper management of product data from both backend and frontend.

V1 >> Role-Based Access Control Problem

There was a security issue where normal users were able to access admin features such as orders, user lists, and admin panels.

This was fixed by implementing proper role-based access control. Now, only users with admin roles can access admin-related pages and actions, while normal users are restricted to their allowed features.

V2 >> Environment Configuration and the hardcode JWT

The project was improved by adding better environment variable handling using dotenv. Now the application loads configuration from a `.env` file when available.

Previously, the **JWT secret** was hardcoded directly inside the application. This was a serious security risk because if someone gained access to the source code, they could easily retrieve the secret and create or forge valid authentication tokens. This could have allowed an attacker to impersonate users, bypass authentication, and gain unauthorized access to protected data or admin features.

To fix this, the system now checks whether a JWT secret exists in the environment and whether it is strong enough. If the secret is missing or too weak, the application either logs a warning or generates a secure random fallback secret using cryptographic functions.

V3 >> DDoS Protection and Rate Limiting Improvements

We also checked the application's protection against DDoS-style attacks and high traffic by reviewing and strengthening existing rate limiting mechanisms. We did the following to fix that:

- **App-level rate limiting (express-rate-limit)**
  - File: `rateLimit.ts`
  - `globalLimiter`: 100 requests per 15 minutes per IP (applied globally)
  - `authLimiter`: 10 requests per 15 minutes per IP (applied to `/auth` routes)
- **Rate limiter applied in the app**
  - File: `index.ts`
  - `app.use(globalLimiter)` and `app.use('/auth', authLimiter, routes)`

Nginx proxy-level limits

    - File: `nginx.conf`
    - Config included `limit_req_zone` and `limit_req` / `limit_conn` directives to control request rates and connections at the proxy level

Without this we could have faced the issues like:

- Request flooding (API overload)
- Authentication brute-force attacks on `/auth`
- Resource exhaustion (CPU/memory/network saturation)
- Service slowdown or complete downtime

V4 >> CORS Security Fix

We also noticed that any link can get into the backend because there were no params for cors()
So, we added whitelist-based policy so that only the allowed URLs can pass this, otherwise any website (like `http://evil.com`) could send requests to your backend API.
And the browser would not block these requests because CORS would allow all origins.

V5 >> Password Security Improvements

Password security was weak at the start, allowing very simple passwords.

This was improved by increasing password requirements. Now passwords must be at least 8 characters long and include uppercase and lowercase letters.

Bcrypt hashing strength was also increased to make password cracking harder.

If that was allowed:

- **Easy brute-force attacks**
    - Attackers could quickly guess passwords using automated tools.
- **Dictionary attacks**
    - Common passwords would be cracked almost instantly.
- **Account takeover**
    - Attackers could log in as real users with very little effort.
- **Admin compromise risk**
    - If an admin used a weak password, the entire system could be exposed.

And if bcrypt was not strengthened:

- Password hashes would be easier to crack offline.
- Attackers who obtained the database could use GPU/CPU tools to recover passwords faster.
- Lower salt rounds is like faster cracking attempts.

V6 >> Hardcoded DB password and username

We also noticed that the DB credentials were hardcode in the application, so to fix it we

- Added the DB credentials into the .env file.
- Ensured `config.ts` loads environment variables and enforces JWT secret validation with a secure fallback.
- Confirmed `ormconfig.js` uses environment variables and fails if DB credentials are missing in production.
- Updated `docker-compose.yml` to pass environment variables properly into the API container.

And if we haven't fix it we could face these kind of issues:

- Exposure of DB username and password in the codebase
- Unauthorized access to the database
- Risk of data theft, modification, or deletion
- Difficult and unsafe credential management across environments
- Secrets potentially being exposed inside Docker images

V7 >> Authentication Token Security

Currently, authentication tokens are being stored in localStorage or cookies accessible from JavaScript. This is not very secure because XSS attacks could steal tokens.

The recommended fix is to use HttpOnly cookies so tokens cannot be accessed from JavaScript, along with better CSRF protection and short-lived access tokens.

Also we can further improve this security risk by enforcing HTTPS, tightening CORS, and improving frontend security to reduce XSS risks.